

Lunettes Connectées

Rapport de projet Java Avancé

Rosina José

Jung Maxime

Garcia Léon

Holder Jules

Master 1 MIAGE

Java Avancé - M. Jérôme Bertrand

IDMC 2025/2026

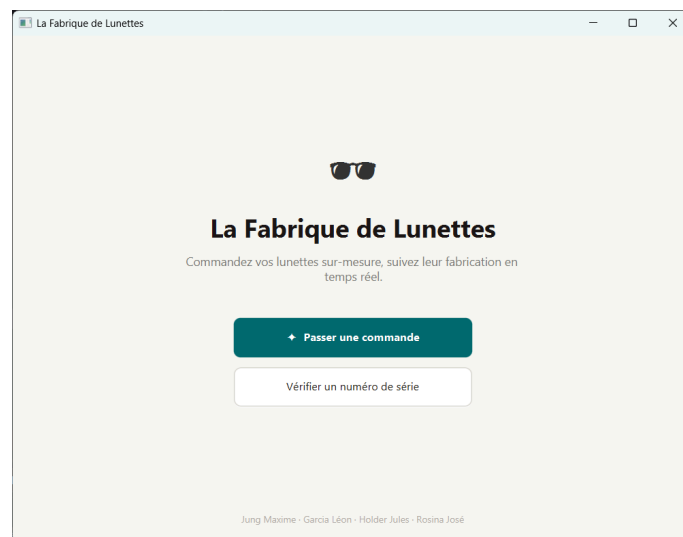
Introduction	3
Contexte	3
MQTT	4
Architecture du projet	5
Outils et technologies utilisés	6
Common	7
Principales méthodes	7
Frontend	9
L'écran d'accueil	9
Le catalogue de commande	10
Le suivi de fabrication	10
Vérification du numéro de série	10
Amélioration UX et UI	11
Backend	13
Usine	13
Mosquitto	14
Principales méthodes	17
Cycle de vie d'une commande	19
Tests	20
Usine	20
valider()	20
produire()	20
ajouterCommandeMutualisee()	21
getSerialsProduits()	22
Test du Backend	22
CommandeProcessingIntegrationTest	22
MosquittoBackTest	23
Common	23
CommandeSerializerTest	23
LivraisonSerializerTest	24
MosquittoProtocolTest	25
Publication	26
Utilisation	27
Difficultés rencontrées	27
Perspective future	28
Réponse aux questions	28
Conclusion	29

Introduction

Dans le cadre de notre M1 MIAGE pour le cours de Java avancé, nous avons dû réaliser un système distribué servant à commander, fabriquer et délivrer des lunettes connectées qui implémentent l'IA.

Ce projet a été l'occasion pour nous de mettre en application les concepts de programmation avancée vus en cours, comme JavaFX et les threads, ainsi que de découvrir par nous même de nouveaux concepts et outils, comme le broker MQTT Mosquitto.

Dans ce rapport nous présenterons l'architecture de notre projet ainsi que notre parcours tout au long de ce développement.



Page d'accueil de notre application

Notre groupe est constitué de ROSINA José, JUNG Maxime, HOLDER Jules et GARCIA Léon.

Contexte

Le mécanisme de fabrication et de livraison des lunettes est structuré en plusieurs étapes et éléments :

Le système doit impérativement être séparé en deux sous-systèmes communiquant de manière asynchrone via le protocole MQTT.

Le projet doit être "Production Ready" :

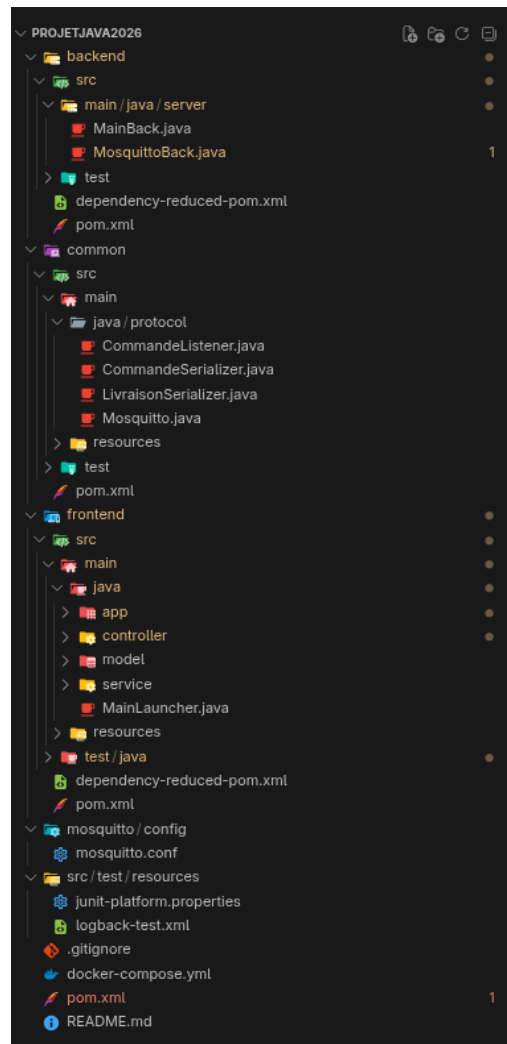
- **Gestion des erreurs** : Le système doit anticiper et gérer élégamment les pertes de connexion MQTT, les commandes invalides, et les pannes lors de la fabrication.
- **Configuration externalisée** : Les variables d'environnement (comme l'adresse du broker MQTT) doivent être lues depuis un fichier de configuration dédié.
- **Traçabilité** : Une implémentation de logs propres et pertinents est exigée (ni trop verbeux, ni insuffisants).
- **Gestion des dépendances** : Le projet doit utiliser Maven. L'usine doit elle-même être packagée sous forme de composant indépendant.

MQTT

MQTT (Message Queuing Telemetry Transport) est un protocole de messagerie léger basé sur un modèle de Publication/Abonnement, qui s'appuie sur un intermédiaire central appelé broker (dans notre cas, Mosquitto). Dans le cadre de ce projet, son intérêt est majeur : il permet d'architecturer notre application comme un système distribué orienté événements et totalement asynchrone. Grâce à ce protocole, le Frontend avec le client JavaFX, et le Backend notamment l'Usine sont parfaitement découplés.

Ainsi, lorsqu'un client passe commande, l'interface graphique publie simplement la demande sur le bus de données sans rester bloquée en attendant la fabrication. Elle s'abonne ensuite à un canal de réponse pour être notifiée de la livraison. Cette approche asynchrone est idéale pour gérer de multiples clients en parallèle tout en garantissant la fluidité et la scalabilité de notre système.

Architecture du projet



Capture d'écran de l'arborescence de notre projet

Notre projet comporte donc deux modules principaux, *backend* et *frontend*. Ceux-ci utilisent tous deux un autre module, *common*, qui permet de stocker les différentes parties des projets communes aux deux, telles que la base du client MQTT pour communiquer entre les deux parties ou bien le sérialiseur de données..

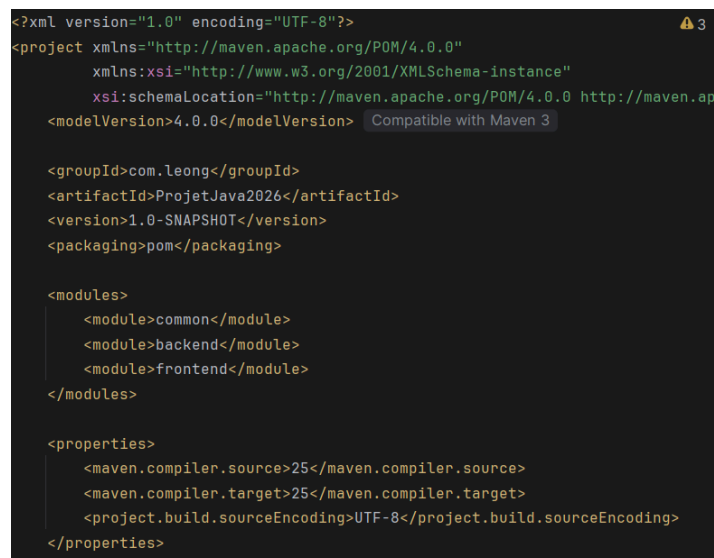
La partie Usine de l'application, qui comprend simplement la classe métier *Usine*, a été placée dans un autre dépôt GitHub, appelé et déployée en un package Maven via une GitHub Action, utilisé dans le projet. Ce deuxième projet s'appelle *lunettes-factory*,

Nous avons donc utilisé Maven pour gérer les dépendances du projet, comme demandé. Un premier fichier *pom.xml* est situé à la racine du projet, ainsi que 3 autres, situés dans chaque module, *frontend*, *backend* et *common*. Nous avons également un dossier *mosquitto* et un fichier *.properties*, comprenant les données nécessaires aux clients Mosquitto. Enfin, un fichier *.gitignore* nous sert à indiquer

quels fichiers ne doivent pas être envoyés dans le dépôt GitHub, tels que les dossiers *target* ou bien les informations relatives aux clients Mosquitto créés générés à chaque lancement de l'application.

Outils et technologies utilisés

Pour réaliser ce projet, nous avons donc utilisé le langage de programmation Java pour la partie BackEnd en version 21; ainsi que JavaFX en version 21 pour la partie FrontEnd. L'ensemble de nos dépendances pour le projet, que ce soit pour les tests, le broker Mosquitto ou bien la version de Maven, est disponible dans le fichier pom.xml à la racine du projet. Nous y avons aussi indiqué le chemin pour accéder au dépôt *fabricateur* pour compléter la partie Usine qui crée les lunettes. L'utilisation de Maven facilite grandement la gestion de nos dépendances et nous permet d'avoir tous les mêmes versions.

A screenshot of a code editor showing the beginning of a Maven pom.xml file. The code is in XML format and includes standard Maven project declarations such as groupId, artifactId, version, and packaging. It also lists three modules: common, backend, and frontend. Properties for the Maven compiler and source encoding are also defined. A warning icon in the top right corner indicates a compatibility issue with Maven 3.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.ap
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.leong</groupId>
  <artifactId>ProjetJava2026</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>pom</packaging>

  <modules>
    <module>common</module>
    <module>backend</module>
    <module>frontend</module>
  </modules>

  <properties>
    <maven.compiler.source>25</maven.compiler.source>
    <maven.compiler.target>25</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
```

Capture d'écran du début de notre fichier pom.xml à la racine du projet

De plus notre projet est structuré en plusieurs modules Maven qui gèrent chacun un pan de notre application :

- *frontend*, comprenant l'application client en JavaFX, qui communique avec le backend via le broker Mosquitto
- *backend*, comprenant la partie serveur de Mosquitto, ainsi que l'utilisation de l'Usine dans le Main du back
- *common*, comprenant des classes utilisées à la fois dans *backend* et *frontend*; telles que les serializers de données ou bien la classe parent Mosquitto

Concernant les environnements de travail, seul Jules a utilisé Visual Studio Code, tandis que Léon, Maxime et José ont tous les 3 utilisé IntelliJ; mais cela dépendait simplement des préférences de chacun.

L'ensemble de notre projet est disponible dans un dépôt GitHub; et nous avons majoritairement utilisé Discord pour communiquer afin de travailler sur ce projet, auquel s'ajoutent quelques réunions en présentiel à l'IDMC.

Common

Le module common regroupe, dans le package protocol, l'ensemble des classes partagées entre le frontend et le backend.

La classe abstraite *Mosquitto* constitue la base de tout client MQTT : elle gère l'initialisation de la connexion au broker via l'URL lue depuis le fichier `.properties`, instancie le *MqttClient* avec une *MemoryPersistence*, fixe le QoS à 1, et expose les noms des topics de base (`orders/` et `serials/`).

Les deux classes *CommandeSerializer* et *LivraisonSerializer* définissent le format d'échange personnalisé sous la forme `{TYPE:QUANTITE;TYPE:QUANTITE;...}` : *CommandeSerializer* convertit une `Map<TypeLunette, Integer>` en cette chaîne (et inversement, avec validation par expression régulière), tandis que *LivraisonSerializer* fait de même pour une liste de Lunette produites avec leurs numéros de série.

Enfin, l'interface *CommandeListener* définit le contrat d'écoute des événements d'une commande (*onValidated*, *onDelivery*, *onCancelled*, *onError*, *onChecked*) avec des implémentations default vides, permettant à chaque implémenteur de ne surcharger que les méthodes qui le concernent.

Centraliser toutes ces classes dans common garantit que frontend et backend partagent exactement le même format de sérialisation et la même logique de connexion, sans aucune duplication.

Principales méthodes

Nom	Paramètres	Classe	Description
<code>initClient</code>	<code>String client</code>	<i>Mosquitto</i>	Initialise le client MQTT en lisant l'URL du broker depuis le fichier <code>.properties</code> , instancie un <i>MqttClient</i> avec <i>MemoryPersistence</i> , se connecte au

			broker et fixe le QoS à 1. Logue une erreur en cas d'échec d'I/O.
serialize	Map<Fabricateur.TypeLunette, Integer> commande	CommandeSerializer	Convertit une Map de types de lunettes et quantités en chaîne au format {TYPE:QTE;TYPE:QTE;...}.
deserialize	String payload	CommandeSerializer	Parse la chaîne au format {TYPE:QTE;...} en Map. Valide le format via regex et lève une IllegalArgumentException si invalide.
serialize	List<Fabricateur.Lunette> livraisons	LivraisonSerializer	Convertit une liste de lunettes produites en chaîne au format {TYPE:SERIAL;...}.
deserialize	String payload	LivraisonSerializer	Parse la chaîne de livraison et retourne une liste de chaînes lisibles au format "Type: X \\\nSérie: Y".
onValidated	String uuid	CommandeListener	Appelée quand une commande est validée par l'usine. Implémentation default vide.
onDelivery	String uuid	CommandeListener	Appelée quand les lunettes sont livrées avec leurs numéros de série. Implémentation default vide.
	String serials		

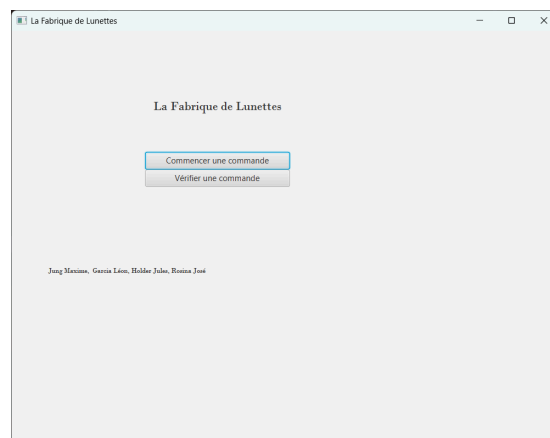
onCancelled	String uuid	CommandeListener	Appelée quand une commande est annulée, avec la raison. Implémentation default vide.
	String reason		
onError	String uuid	CommandeListener	Appelée en cas d'erreur lors de la fabrication. Implémentation default vide.
	String error		
onChecked	String uuid	CommandeListener	Appelée lors de la vérification d'un numéro de série, retourne le type de lunette associé. Implémentation default vide.
	String typeLunette		

Frontend

Notre application est composée de quatre écrans :

L'écran d'accueil

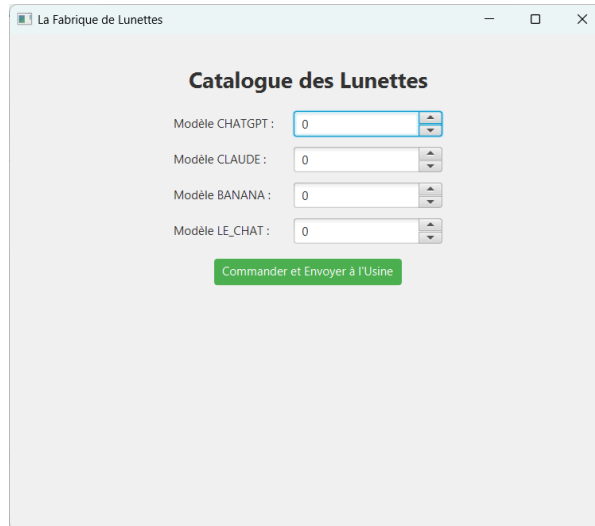
Au lancement de l'application, l'écran d'accueil s'affiche avec les deux boutons permettant d'aller sur les deux cas d'utilisation :



Capture d'écran de la première version de l'accueil

Le catalogue de commande

Quand on clique sur “Commencer une commande” c’est sur cet écran que l’on arrive. On a quatre sélections de quantité, une pour chaque modèle. Les quantités sont limitées de 0 à 9 pour coller aux exigences du sujet. On peut ensuite publier cette commande en appuyant sur “Commander...”



La Fabrique de Lunettes

Catalogue des Lunettes

Modèle CHATGPT : 0

Modèle CLAUDE : 0

Modèle BANANA : 0

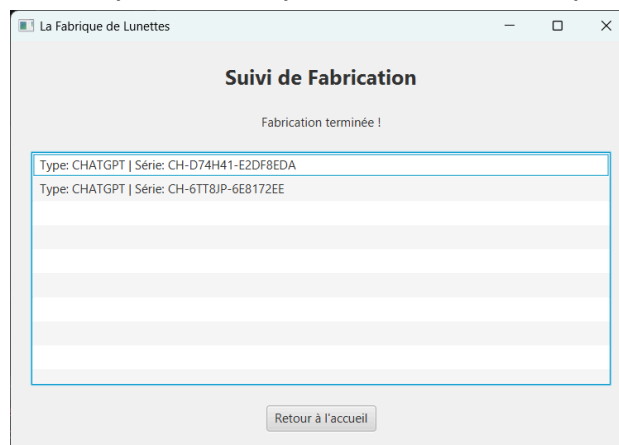
Modèle LE_CHAT : 0

Commander et Envoyer à l'Usine

Capture d'écran de la première version de l'écran de commande

Le suivi de fabrication

Si la commande à bien été envoyée et qu'elle est en cours de production, après un léger temps de chargement les types de lunettes ainsi que leurs numéros de séries sont affichés. Ce qui valide la production et la réception des lunettes.



La Fabrique de Lunettes

Suivi de Fabrication

Fabrication terminée !

Type: CHATGPT Série: CH-D74H41-E2DF8EDA
Type: CHATGPT Série: CH-6TT8JP-6E8172EE

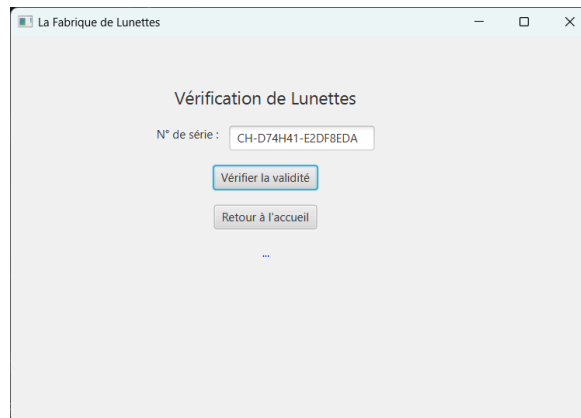
Retour à l'accueil

Capture d'écran de la première version de l'écran de suivi de fabrication

Vérification du numéro de série

L'autre cas d'utilisation permet de vérifier que votre numéro de série est bien attribué à une paire de lunettes. Pour vérifier cela vous pouvez entrer le numéro de

série et après avoir interrogé le backend, on vous indiquera le type de cette lunette ou 'invalid' si ce numéro ne correspond à rien.

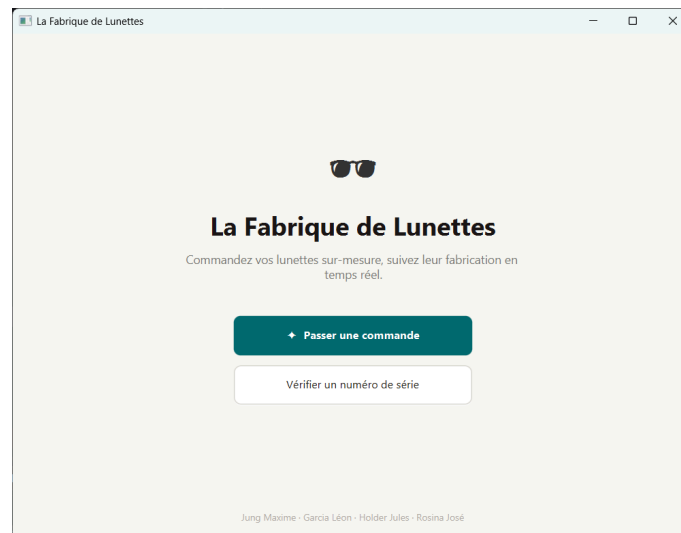


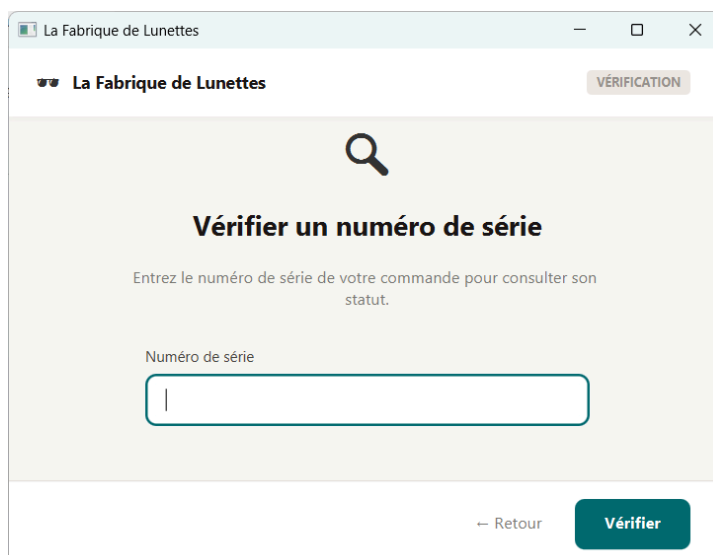
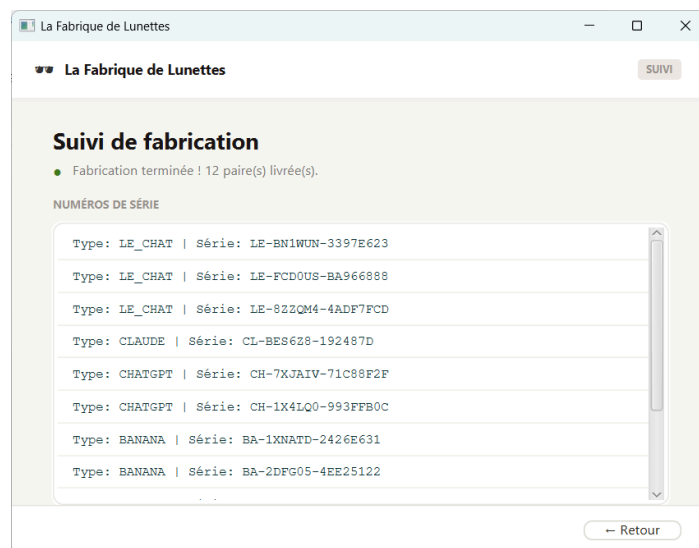
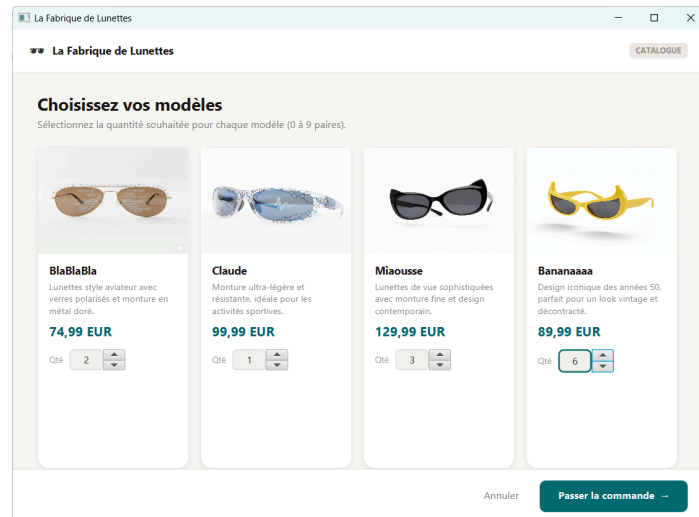
Capture d'écran de la première version de la vérification de numéro de série

Amélioration UX et UI

Ces écrans ont été réalisés uniquement à l'aide de nos petites mains et de nos cours sur JavaFX pour que l'application soit fonctionnelle, l'UX et l'UI n'étant pas nos priorités pour ce projet.

Cependant par pure satisfaction personnelle et parce que nous n'avions plus le temps de refaire l'interface, nous avons décidé d'utiliser l'IA Gemini pour améliorer le visuel de notre application, ce qui donne ces visuels :





Captures d'écrans des versions finales de l'écran d'accueil, de la page de commande, de la page de suivi de fabrication et de la page de vérification de numéro de série de l'application

Backend

Usine

L'usine de la partie Backend est le cœur de cette application. Elle est placée dans un deuxième dépôt GitHub, publié par une GitHub Action en une dépendance Maven. Elle est basée sur la dépendance *Fabricateur*, ajoutée depuis le fichier dépendances pom.xml. Elle est représentée dans notre projet par la classe *Usine*, située dans le package *factory*.

Une *Usine* est matérialisée par un fabricant, issue de la dépendance citée précédemment, un logger qui permet d'enregistrer l'ensemble des événements qui surviennent dans l'usine, une liste de l'ensemble des lunettes produites, un worker service qui va permettre de traiter l'ensemble des commandes et une liste des commandes en attente d'être traitées. Tous ces attributs sont *final*, et dans le cas du logger, il est *static* pour permettre d'être utilisé sans avoir à créer la classe *Usine*.

L'architecture globale de cette classe est axée autour du worker service : un exécuteur mono threadé est instancié à l'initialisation du composant. Il exécute une boucle d'écoute active en tâche de fond sur la file d'attente. L'utilisation d'un thread dédié garantit que les opérations de traitement sur la machine physique ne sont pas interrompues par les cycles de vie des threads de réception.

Les commandes, passées par le client via l'interface JavaFX dédiée, arrivent dans le backend sous forme de messages via le broker avec dans le topic l'UUID de la commande et en payload le contenu de la commande à produire. Une fois le message correctement reçu, le message est parsé pour en extraire une Map contenant les types de lunettes et les quantités commandées associées.

Cette Map passe ensuite par la méthode *valider*, qui s'assure que le contenu de la commande est bien réalisable: il y a bien indication de lunettes à fabriquer et la commande est non nulle et le nombre de chaque type de lunettes à fabriquer est bien compris entre 0 inclus et 9 inclus. Une fois la commande validée, le message de confirmation est envoyé au client via le broker par la méthode *validateOrder* de *MosquitoBack*. Ce n'est qu'à ce moment que l'Usine va réellement recevoir les informations de la commande à produire.

La méthode *ajouterCommandeMutualisée* envoie donc enfin les commandes à l'usine, le worker les place dans les commandes attente et appelle la fonction *traiterCommandesMutualisées*. Cette fonction permet à l'usine de rassembler les données des différentes commandes pour connaître à l'avance le nombre de chaque type de lunettes à produire et donc de simplifier la chaîne de production.

Ensuite, la mutualisation des commandes amène à la production des lunettes pour chaque type différent apparaissant dans la commande mutualisée, donc au maximum 4. Pour chaque type de lunettes, la capacité de production à la chaîne

du fabricant est limitée. Nous nous retrouvons donc dans la situation où l'on ne peut produire à la fois que la valeur minimale entre le nombre de lunettes à produire de ce type dans cette commande mutualisée et la capacité de production par cycle du fabricant.

Cette valeur minimale est donc envoyée en production, dans un bloc `synchronized`, qui permet d'éviter les possibles collisions entre les commandes et que le fabricant ne soit surchargé. Ces instructions sont donc placées dans une boucle `while`, effective jusqu'à ce qu'il ne reste plus de lunettes de ce type à produire. Une fois qu'il n'en reste plus, les lunettes produites sont remontées dans les appels successifs de fonctions, jusqu'à l'étape de distribution des lunettes.

A ce niveau-ci, les lunettes produites sont redistribuées dans chaque commande en fonction du type de lunettes et de la quantité de lunettes nécessaires. Les *CompletableFuture*, instanciées dans la méthode *ajouterCommandeMutualisée*, sont closes à ce moment car la production des lunettes a été terminée avec succès, et comme il s'agit du résultat retourné de la méthode, le résultat est traité dans le callback dans *MosquitoBack*: dans notre cas où aucune erreur ne s'est produite, nous pouvons serialiser la commande via *serialize*, et enfin appeler la méthode *deliverOrder* au topic dédié.

Bien évidemment, il s'agit ici du cas où tout le processus de commande et de fabrication des lunettes se passe correctement. Mais si jamais des erreurs surviennent, nous saurions comment les traiter. En effet, si une erreur est levée entre le moment où l'on parse le payload et le moment où le message de validation de commande a bien été envoyé au client, la commande sera immédiatement annulée. Il y a également le cas où une erreurUsine

L'usine de la partie Backend est le cœur de cette application. Elle est placée dans un deuxième dépôt GitHub, publiée par une GitHub Action en une dépendance Maven. Elle est basée sur la dépendance Fabricateur, ajoutée depuis le fichier dépendances pom.xml. Elle est représentée dans notre projet par la classe Usine, située dans le package factory.

Une Usine est matérialisée par un fabricant, issue de la dépendance citée précédemment, un logger qui permet d'enregistrer l'ensemble des événements qui surviennent dans l'usine, une liste de l'ensemble des lunettes produites, un worker service qui va permettre de traiter l'ensemble des commandes et une liste des commandes en attente d'être traitées. Tous ces attributs sont final, et dans le cas du logger, il est static pour permettre d'être utilisé sans avoir à créer la classe Usine.

L'architecture globale de cette classe est axée autour du worker service : un exécuteur mono threadé est instancié à l'initialisation du composant. Il exécute une boucle d'écoute active en tâche de fond sur la file d'attente. L'utilisation d'un thread

dédié garantit que les opérations de traitement sur la machine physique ne sont pas interrompues par les cycles de vie des threads de réception.

Les commandes, passées par le client via l'interface JavaFX dédiée, arrivent dans le backend sous forme de messages via le broker avec dans le topic l'UUID de la commande et en payload le contenu de la commande à produire. Une fois le message correctement reçu, le message est parsé pour en extraire une Map contenant les types de lunettes et les quantités commandées associées.

Cette Map passe ensuite par la méthode valider, qui s'assure que le contenu de la commande est bien réalisable: il y a bien indication de lunettes à fabriquer et la commande est non nulle et le nombre de chaque type de lunettes à fabriquer est bien compris entre 0 inclus et 9 inclus. Une fois la commande validée, le message de confirmation est envoyé au client via le broker par la méthode validateOrder de MosquittoBack. Ce n'est qu'à ce moment que l'Usine va réellement recevoir les informations de la commande à produire.

La méthode ajouterCommandeMutualisée envoie donc enfin les commandes à l'usine, le worker les place dans les commandes attente et appelle la fonction traiterCommandesMutualisées. Cette fonction permet à l'usine de rassembler les données des différentes commandes pour connaître à l'avance le nombre de chaque type de lunettes à produire et donc de simplifier la chaîne de production.

Ensuite, la mutualisation des commandes amène à la production des lunettes pour chaque type différent apparaissant dans la commande mutualisée, donc au maximum 4. Pour chaque type de lunettes, la capacité de production à la chaîne du fabricant est limitée. Nous nous retrouvons donc dans la situation où l'on ne peut produire à la fois que la valeur minimale entre le nombre de lunettes à produire de ce type dans cette commande mutualisée et la capacité de production par cycle survient dans l'usine et la commande ne peut être complétée: s'il y a une interruption dans un thread ou un problème lors de la mutualisation des commandes. Dans ce cas-ci aussi, l'erreur sera transmise au client qui pourra savoir pourquoi sa demande n'a pas pu être satisfaite.

Mosquitto

Pour le développement de la partie communication entre l'usine et l'application, nous devons nous baser sur le broker de messages mqtt *Mosquitto*, avec une configuration minimale, sans question de connexion ou d'authentification et une utilisation du port par défaut (1883).

Tout d'abord, nous avons cherché à nous renseigner sur Mosquitto et voir comment il pouvait être implémenté dans un projet en java. En suivant un guide

(<https://www.cedalo.com/blog/mqtt-in-java-guide>), nous avons pu tester rapidement Mosquitto et voir comment gérer les paramètres de base, la gestion des erreurs ou de l'arrivée des messages. Pour ajouter un système de connexion client à un serveur Mosquitto dans un projet Java avec Maven, il faut ajouter une dépendance dans le pom.xml. Il faut également soit avoir Mosquitto installé sur son ordinateur, soit le faire tourner dans un conteneur sur Docker par exemple, comme nous verrons par la suite.

Après avoir réussi à lancer le programme proposé dans le guide, nous avons pu directement implémenter ce module de communication dans notre projet. Pour cela nous avons créé une branche dédiée et avons pu commencer à travailler concrètement sur cette partie.

Au début, nous ne savions pas exactement quelle architecture devrait prendre concrètement ceci, sachant qu'il nous faut créer deux clients, un pour la partie Usine (dans le *backend*) et un autre pour la partie application (dans le *frontend*), qui devront fonctionner simultanément.

Nous avons donc décidé de créer une classe parent, *Mosquitto*, qui permet seulement d'initier un nouveau client avec un nom qui lui est passé en paramètre, l'adresse du broker spécifiée dans le fichier .properties, les noms des topics de base, ainsi que le QoS du client. Ce QoS vaut 1, qui correspond au mode de transfert par défaut, signifie que tant que l'expéditeur n'aura pas reçu d'accusé de réception de l'envoi de son message, celui-ci sera renvoyé jusqu'à bonne réception.

Ensuite, pour la partie usine, nous avons donc créé une classe enfant, *MosquittoBack*, qui permet d'initier le client avec un nom spécifique. Une fois que ce client est connecté, nous pouvons ajouter des callback qui permettront de gérer les différents événements qui peuvent se produire lors de la réception ou de l'envoi de messages, en faisant un Override des méthodes déjà existantes. Ici, les méthodes *connectionLost* et *deliveryComplete* se contentent simplement de print ce qu'il s'est passé. La méthode *messageArrived* s'occupe de rediriger un message reçu vers le traitement approprié selon son topic et son ID.

Ces redirections passent d'abord par des appels à l'usine pour lancer les productions de commandes, et ensuite font appel aux autres méthodes du client *MosquittoBack*. Ces méthodes correspondent aux différents événements et routes des topics de cette application: *validateOrder* pour valider une commande, *cancelOrder* pour annuler une commande, *deliverOrder* pour livrer une commande, *errorOrder* lorsqu'une erreur survient lors de la production d'une commande, *serialsInfos* qui renvoie les informations d'un produit lorsqu'elles sont demandées par le client.

Pour la partie application frontend, nous avons également créé une classe enfant, *MosquittoApp*, qui reprend la même structure que *MosquittoBack*. Ici, la méthode *messageArrived* doit gérer les différents événements qui se produisent

après que ce client ait passé commande auprès de l'Usine. Il peut également réceptionner les informations d'un produit après qu'il les ait demandées. Les méthodes propres à *MosquittoApp* sont donc celles pour passer commande, *order*, en passant en corps de message le contenu de la commande; et celle pour demander des informations sur un produit, *askSerials*.

Les topics ici sont classés en deux catégories spécifiques, avec des chemins indépendants:

- Les topics commençant par *order* concernent les commandes passées par l'application, et les différentes réponses possibles par l'usine. A chaque commande est associé un ID unique généré par l'application qui permettra d'identifier la commande lors de son processus de fabrication.
- Les topics commençant par *serials* concernent les demandes d'informations d'un produit par l'application et la réponse de l'usine. Seulement deux types de messages se font donc sur ce topic

Le problème avec l'utilisation de Mosquitto en tant que broker de messages dans notre utilisation est qu'il est nécessaire d'avoir un broker qui tourne sur le PC qui lance l'application. Afin de faciliter cette utilisation, nous avons fait le choix de conteneuriser Mosquitto via Docker. Nous avons ainsi un fichier *docker-compose.yml* qui utilise simplement un seul service, *mqtt-broker*, basé sur l'image *eclipse-mosquitto:latest*. Ce service précise que le port utilisé est 1883, et qu'il doit en permanence être relancé si celui-ci se stoppe. Pour le lancer, il suffit d'avoir docker de lancé sur son PC et de lancer la commande *docker-compose up* dans un terminal pour que le service soit lancé et que la communication entre les deux clients Mosquitto se fasse correctement.

Nous avons rencontré très peu de difficultés et de questionnements lors du développement de cette partie. Certaines concernaient la gestion des topics, et plus spécifiquement le fait qu'il nous a fallu placer la souscription à un topic avant la publication d'un message, car cela pourrait poser problème si l'on souscrit à un topic alors que le message précédent ne soit pas encore arrivé au destinataire. Cela concerne ici les méthodes *order* et *askSerials* dans le client *MosquittoApp*.

Nous avons aussi pendant longtemps pensé qu'il était nécessaire pour *MosquittoBack* de devoir *unsubscribe* au topic *orders* à chaque fois qu'un processus de commande prenait fin, que ce soit lors d'une erreur ou d'une annulation; ou bien lorsque l'on envoyait les données de lunettes après que le client en ait fait la demande sur le topic *serials*. Nous nous sommes rendus compte par la suite qu'en réalité cela ne servait à rien du tout car ce client Mosquitto est simplement abonné à un topic wildcard, finissant par *+* ou *#*. Ce genre de topic nous permet d'être abonné à un topic de base et de recevoir tous les messages sur ce topic sur un seul niveau (*#*) ou sur plusieurs niveaux (*+*). De plus, comme les seuls messages que peut recevoir *MosquittoBack* sont les nouvelles demandes de commande et les

demandes d'informations sur des lunettes, il n'a pas besoin d'attendre de nouveaux messages dépendant des messages qu'il a déjà reçus.

Principales méthodes

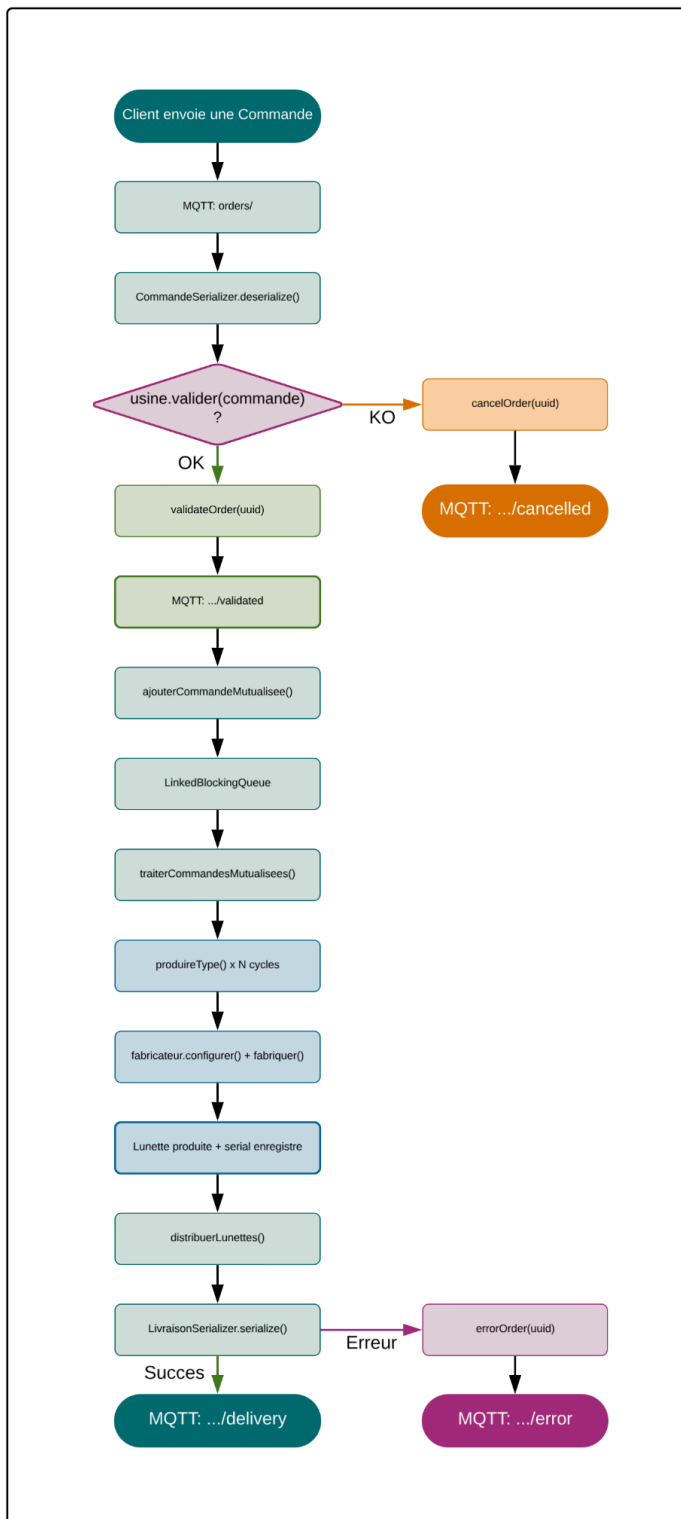
Les méthodes principales dans notre backend sont répertoriées ci dessous :

Nom	Paramètres	Classe	Description
message Arrived	String topic	MosquittoBack	Traite les messages qui arrivent sur le broker MQTT et utilisent les autres méthodes pour y répondre.
	MqttMessage message		
validateOrder	String uuid	MosquittoBack	Publie le topic de validation d'une commande.
cancelOrder	String uuid	MosquittoBack	Publie le topic d'annulation d'une commande.
deliverOrder	String uuid	MosquittoBack	Publie le topic pour dire que les lunettes sont livrées.
errorOrder	String uuid	MosquittoBack	Publie le topic qui présente l'erreur si il y en a une.
produire	Map<Fabricateur.TypeLunette, Integer> typesLunettes	Usine	Itère sur chaque type de lunettes de la commande et délègue la production à produireType() point d'entrée pour la production séquentielle.
valider	Map<TypeLunette, Integer> typesLunettes	Usine	Vérifie que la commande est non vide, que la quantité totale est > 0 et que chaque quantité est comprise entre 0 inclus

Nom	Paramètres	Classe	Description
			et 9 inclus. Lève une <code>IllegalArgumentException</code> sinon
ajouterCommandeMutualisee	String uuid	Usine	Crée un <code>CompletableFuture</code> , encapsule la commande dans un <code>CommandeEnAttente</code> et la dépose dans la <code>LinkedBlockingQueue</code> pour traitement asynchrone
	Map<TypeLunette, Integer> commande		
traiterCommandesMutualisees	List<CommandeEnAttente> commandes	Usine	Agrège toutes les commandes en attente en une seule Map mutualisée, appelle <code>produire()</code> une seule fois, puis délègue la distribution via <code>distribuerLunettes()</code> . En cas d'erreur, complète tous les futures en exception.
produireType	TypeLunette type	Usine	Produit un type de lunettes par cycles limités par la capacité du fabricant (<code>getCapacity()</code>), en remplissant un tableau d'emplacements et en appelant <code>fabriquer()</code> dans un bloc <code>synchronized</code> sur le fabricant
	int quantiteTotale		
distribuerLunettes	List<CommandeEnAttente> commandes	Usine	Organise les lunettes produites par type dans des <code>Queue</code> , puis distribue à chaque commande ses lunettes correspondantes et complète son <code>CompletableFuture</code>

Nom	Paramètres	Classe	Description
	List<Lunette> lunettesProduites		

Cycle de vie d'une commande



Nous avons représenté le cycle d'une vie d'une commande au sein de notre projet par un diagramme d'activité. De cette manière, nous pouvons voir son évolution entre le moment où le client passe commande et le moment où soit il la reçoit, soit une erreur survient ou soit la commande est annulée.

Tests

Pour faire nos tests nous avons utilisé JUnit et Mockito pour isoler les composants et garantir la fiabilité de nos tests.

Usine

valider()

Nom du test	Description	Résultat attendu
valider_commandeVide_levException	Commande avec une Map vide	IllegalArgumentException levée
valider_quantitesInvalides_levException	Quantités invalides : -5, -1, 0, 10, 15 (test paramétré)	IllegalArgumentException levée pour chaque valeur
valider_quantiteSupA9_levException	Quantité = 10 (hors borne)	IllegalArgumentException levée
valider_quatreTypesValides_nepasLeverException	Commande avec les 4 types (CHATGPT:2, BANANA:1, CLAUDE:3, LE_CHAT:1)	Aucune exception
valider_quantiteMax_nepasLeverException	Quantité = 9 (borne maximale valide)	Aucune exception

produire()

Nom du test	Description	Résultat attendu
produire_chatgpt_retourneNombreCorrect	Commande de 3 CHATGPT	Liste de 3 lunettes
produire_banana_retourneNombreCorrect	Commande de 2 BANANA	Liste de 2 lunettes

produire_leChat_retourneNombreCorrect	Commande de 1 LE_CHAT	Liste de 1 lunette
produire_quatreTypes_retourneTotalCorrect	Commande des 4 types (total 6)	Liste de 6 lunettes
produire_commandeVide_retourneListeVide	Map vide passée à produire()	Liste vide retournée
produire_maxQuantiteParType_fabriqueCorrectement	9 BANANA (quantité max)	Liste de 9 lunettes
produire_depasseCapacite_fabriqueEnPlusieursLots	Capacité mock = 2, commande de 5 CHATGPT	5 lunettes produites en plusieurs lots ; configurer() appelé au moins une fois
produire_enregistreSerialsProduites	Production de 2 CLAUDE	getSerialsProduits() contient 2 entrées
produire_deuxAppels_accumuleLesSerials	2 CHATGPT puis 1 BANANA	getSerialsProduits() contient 3 entrées au total

ajouterCommandeMutualisee()

Nom du test	Description	Résultat attendu
ajouterCommandeMutualisee_commandeValide_retourneCompletableFuture	Commande valide soumise en asynchrone	CompletableFuture non null, non en erreur, résultat récupérable via .get()
ajouterCommandeMutualisee_multipleCommandes_traiteEnsemble	Deux commandes soumises simultanément (uuid-1 et uuid-2)	Les deux futures sont non null après 100ms d'attente

ajouterCommandeMutualisee_apresShutdown_neBlokkePas	Commande soumise après shutdown()	Future non null, pas de blocage
---	-----------------------------------	---------------------------------

getSerialsProduits()

Nom du test	Description	Résultat attendu
getSerialsProduits_sansFabrication_retourneMapVide	Aucune production effectuée	Map vide
getSerialsProduits_apresProduction_contientTousLesSerials	Production de 3 BANANA	Map de 3 entrées, chaque serial contient "BANANA", chaque type est BANANA

Test du Backend

CommandeProcessingIntegrationTest

Nom du test	Description	Résultat attendu
fullCommandFlow_endToEnd	Flux complet : sérialiser → produire → sérialiser la livraison (CHATGPT:2 + CLAUDE:1)	3 lunettes produites, sérialisées et désérialisées correctement
commandValidationAndProduction_withMultipleTypes	4 types commandés (total 8 lunettes)	8 lunettes, round-trip sérialisation/désérialisation OK
invalidCommand_shouldFailValidation	Quantité -1	IllegalArgumentException levée à la validation
invalidSerializedCommand_shouldFailDeserialization	Payload "INVALID:DATA" (sans ; final)	IllegalArgumentException lors de la désérialisation

commandRoundtrip_shouldMaintainData	Sérialisation puis désérialisation d'une commande	Map désérialisée identique à l'originale
serialTracking_allProducedUnitsShouldBeTracked	Commande CHATGPT:3 + BANANA:2	getSerialsProduits() contient 5 entrées, tous de type CHATGPT ou BANANA
capacityLimits_shouldHandleProductionBeyondCapacity	Capacité = 2, commande de 5 CLAUDE	5 lunettes produites, configurer() vérifié via Mockito

MosquittoBackTest

Nom du test	Description	Résultat attendu
payload_valide_deserialiseEtTransmisAUsine	Payload "CHATGPT:2;CLAUDE:1;" désérialisé et transmis à valider() et ajouterCommandeMutualisee()	Les deux méthodes de l'Usine sont bien appelées (vérification Mockito)
payload_invalide_leveExceptionALaValidation	Commande avec quantité -1	IllegalArgumentException levée par valider()
getSerialsProduits_serialConnu_retourneType	Serial "SERIAL-001" associé à BANANA	Retourne la chaîne "BANANA"
getSerialsProduits_serialInconnu_retourneNull	Serial "INCONNU" absent de la map	Retourne la chaîne "null"

Common

CommandeSerializerTest

Nom du test	Description	Résultat attendu
serialize_unType_formatCorrect	Séréalise {CHATGPT: 3}	"CHATGPT:3;"
serialize_commandeVide_retourneChaine	Séréalise une Map vide	Chaîne vide ""
serialize_maxQuantity_retourneFormatCorrect	Séréalise {BANANA: 9}	"BANANA:9;"
deserialize_deuxTypes_retourneMapCorrect	Déséréalise "CHATGPT:3;CLAUDE:2;"	Map avec CHATGPT→3 et CLAUDE→2
deserialize_quatreTypes_retourneTousLesTypes	Déséréalise les 4 types	Map avec toutes les valeurs correctes
deserialize_quantiteZero_retourneZero	Payload "CHATGPT:0;"	Map avec CHATGPT→0
deserialize_formatInvalide_levelIllegalArgumentExcption	Payload "formatInvalide"	IllegalArgumentExcption
deserialize_chaineVide_levelIllegalArgumentExcption	Payload "" vide	IllegalArgumentExcption
deserialize_typeInvalide_levelException	Payload "INVALID_TYPE:5;"	IllegalArgumentExcption
deserialize_quantiteNonNumerique_levelException	Payload "CHATGPT:abc;"	IllegalArgumentExcption
deserialize_missingSeparator_levelException	Payload "CHATGPT2;" (sans :)	IllegalArgumentExcption
deserialize_withBlankParts_levelExceptionSiDoublePointVirgule	Payload "CHATGPT:2;;CLAUDE:1;"	IllegalArgumentExcption
serialize_puisDeserialize_estIdempotent	Round-trip sur les 4 types	Map désérialisée identique à l'originale

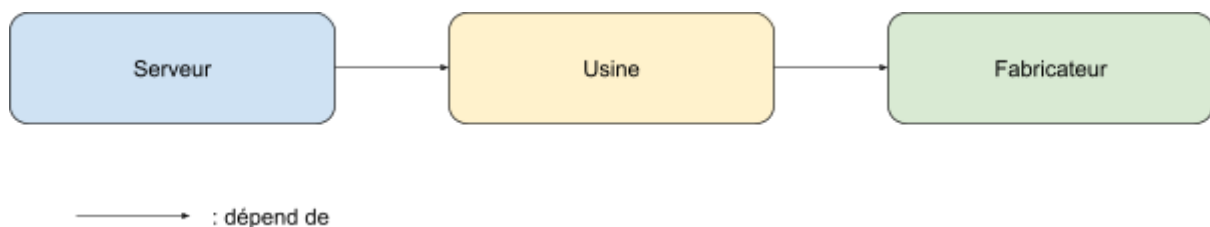
Nom du test	Description	Résultat attendu
serialize_lunettes_formatCorrect	Séréalise 2 lunettes (CHATGPT/S001, CLAUDE/S002)	"CHATGPT:S001;CLAUDE:S002;"
serialize_emptyList_retourneChainVide	Séréalise une liste vide	Chaîne vide ""
serialize_multipleLunettes_maintainOrder	3 BANANA (BAN0, BAN1, BAN2)	La chaîne contient les 3 entrées dans l'ordre
deserialize_quatreTypes_retourneListeComplete	Payload avec 4 lunettes	Liste de 4 éléments au format "Type: X \ Série: Y"
deserialize_unSeulElement_leChat	Payload "LE_CHAT:SER999;"	"Type: LE_CHAT \ Série: SER999"
deserialize_banana_formatCorrect	Payload "BANANA:BAN001;"	"Type: BANANA \ Série: BAN001"
deserialize_payloadVide_retourneListeVide	Payload ""	Liste vide, non null
deserialize_withBlankParts_ignoresBlankParts	Payload "CHATGPT:S001;;CLAUDE:S002;"	2 éléments (la partie vide est ignorée)
deserialize_serialWithSpecialChars_retourneFormatCorrect	Serial "SER-123-ABC" avec tirets	"Type: LE_CHAT \ Série: SER-123-ABC"
serialize_puisDeserialize_estIdempotent	Round-trip sur 2 lunettes	2 éléments, contenant respectivement "CHATGPT" et "BANANA"

MosquitoProtocolTest

Nom du test	Description	Résultat attendu
topicNewOrder_estBienDefini	Vérifie que topicNewOrder est défini, non vide et se termine par "/"	Assertion vraie

Nom du test	Description	Résultat attendu
topicSerials_estBienDefini	Vérifie que topicSerials est défini et non vide	Assertion vraie
qos_valeurValide	Vérifie que le QoS est compris entre 0 et 2 inclus	$0 \leq qos \leq 2$
topicNewOrder_concatenationUuid_valide	Concatène topicNewOrder + un UUID fictif, vérifie que le topic contient / et se termine par l'UUID	Assertion vraie
topicSerials_checkTopic_valide	Construit un topic serials/{uuid}/check, vérifie qu'il contient l'UUID et se termine par "/check"	Assertion vraie

Publication



La particularité de l'usine est qu'elle est séparée du reste de l'application et du dépôt principal. Pour pouvoir y accéder depuis ce même dépôt principal, il est donc nécessaire d'utiliser l'usine en tant que dépendance Maven.

Ainsi, nous avons mis en place une GitHub Action, directement depuis l'interface web GitHub de *lunettes-factory*. Celle-ci a ajouté au projet un fichier `.github/workflows/maven-publish.yml` qui va détailler la démarche à suivre.

La première partie, *on*, va configurer les trois cas où un déploiement sera effectué: lorsqu'un push est réalisé sur la branche *main*, lorsqu'une nouvelle release du projet est publiée ou alors manuellement depuis la page *Action* du projet sur GitHub.

Ensuite, pour le job s'exécute sur une machine virtuelle vierge sous Ubuntu, les permissions de cloner et lire le code source ainsi que publier des packages sur

GitHub Packages sont déclarées explicitement. Le code est récupéré avec `actions/checkout@v4`, puis l'environnement Java 21 est installé pour compiler le code.

C'est à ce moment qu'est généré dynamiquement `settings.xml`: en effet comme nous utilisons deux dépôts distants sécurisés, nous avons besoin des identifiants pour accéder aux deux. Le premier, appelé `github`, correspond à la dépendance `bernard-flou:fabricateur` et utilise les paramètres déjà fournis. Le deuxième, `github-lunettes`, va lui utiliser un token généré sur GitHub pour s'authentifier dans `distributionManagement` du `pom.xml`.

Enfin, le workflow exécute la commande de build finale `mvn -B clean deploy`: en mode non-interactif et après avoir nettoyé le répertoire `target` pour qu'aucun résidu d'un précédent build ne soit encore présent, les sources sont compilées, les ressources sont traitées, les tests sont exécutés, les fichiers JAR sont packagés. Ce n'est qu'à ce moment que le JAR, le JAR avec dépendances et le `pom.xml` sont envoyés sur GitHub Packages sous l'identité `com.lunettes-factory:lunettes-factory:1.0-SNAPSHOT`. C'est ce package qui est donc utilisé dans le backend.

Utilisation

Les informations pour installer, télécharger et utiliser l'application sont disponibles dans le fichier [README.md](#). Nous avons pris le choix de permettre aux utilisateurs d'utiliser Docker pour lancer le broker Mosquitto sur leur machine, mais dans le cas où Mosquitto est déjà installé sur l'ordinateur, il n'est pas nécessaire d'utiliser le fichier `docker-compose.yml`. Il suffit simplement de s'assurer que le service Mosquitto tourne sur l'ordinateur au moment où l'on souhaite lancer l'application java.

Difficultés rencontrées

Pendant ce projet nous avons rencontré de nombreuses difficultés et notamment nos problèmes ont majoritairement été en lien avec Maven. Bien que ce soit un outil permettant d'unifier les versions au sein de notre équipe nous nous sommes plusieurs fois retrouvés face à des commits qui fonctionnent chez l'un mais pas chez les autres à cause d'un cache corrompu ou d'un jdk installé dans la mauvaise version etc... Nous avons pu à chaque fois régler ces erreurs mais ces problèmes nous auront fait perdre en tout beaucoup de temps.

Pour ce qui est du BackEnd et de la logique métier, tous les soucis ont pu être résolus à force d'essais et de corrections, en utilisant des "sout" pour débbugger.

Perspectives futures

Notre frustration lors de ce projet aura été de ne pas avoir pris le temps d'aller au delà de ce qui était attendu, nous aurions notamment aimé mettre en place la fonctionnalité, pour le client, de pouvoir suivre l'avancement de production de sa commande, grâce au sous-topic `order/xxx/status`, comme précisé dans le sujet. Malheureusement, par manque de temps nous n'avons pas pu ajouter cette fonctionnalité qui aurait rendu l'application un peu plus informative pour le client.

Réponse aux questions

Le sujet comporte également une partie nous demandant de répondre à 3 questions concernant notre projet et son développement

1) Comment gérer l'absence d'usine connectée au bus pour produire les lunettes?

Bien que nous ayons paramétré le QoS des clients du broker à 1, ce qui signifie que le protocole garantit que chaque message soit délivré au moins une fois au broker puis au consommateur, il faudrait que le broker sache qu'il doit retenir les messages pour ce consommateur spécifique. Cela implique donc que l'usine se connecte avec une configuration de sessions durable au niveau du client de messagerie Java.

Une solution plus fiable serait d'utiliser la fonctionnalité Last Will Testament de MQTT: cette feature permet au broker de connaître l'état d'un des clients, ici savoir si l'usine est en ligne et disponible ou non. Ainsi, via un topic spécifique, le broker pourrait avertir un client de la disponibilité ou non de l'usine lorsqu'il publie un message sur un topic. Avec la valeur *retained* à *true* dans le payload de la commande, le broker pourrait savoir que ce message doit être stocké dans le cas où l'usine n'est pas connectée. Et à sa prochaine reconnexion, le broker pourrait donc lui transmettre le message du client si l'usine est bien abonnée à ce topic bien sûr.

2) Que faut-il modifier pour gérer plusieurs commandes en parallèle pour un même client?

Pour répondre à ce problème, il faudrait simplement gérer le cas sur la partie *frontend* de notre projet. En effet, nous avons déjà implémenté le fait que chaque commande est individualisée avec un UUID, et que l'usine peut traiter plusieurs commandes en même temps via son système de threads, de commandes à la chaîne et de commandes mutualisées pour faciliter la production. Il nous faudrait donc simplement permettre à l'utilisateur sur son interface de pouvoir accéder à

nouveau à l'interface de commande. Nous pourrions utiliser une `Map<String,Commande>` pour les stocker, et lui permettre de visualiser l'état d'avancement de chacune de ces commandes avec par exemple le topic bonus que nous n'avons pas ajouté, à savoir celui permettant de connaître en temps réel l'avancement d'une commande.

3) Que faut-il modifier pour pouvoir gérer plusieurs usines?

Pour parvenir à gérer plusieurs usines, il faudrait pouvoir modifier l'architecture actuelle des topics, car chaque usine recevrait toutes les commandes passées et les traiterait en même temps. Nous pourrions donc ajouter des topics spécifiques aux usines pour qu'un dispatcher distribue à chaque usine une commande à traiter, par exemple *usine/x/orders/xxx*.

Toutefois, un problème surviendrait si un client souhaite avoir des informations sur un numéro de série d'une paire de lunettes via le topic *serials/xxx/check*, car le dispatcher ne saurait pas à quelle usine adresser cette requête. Il faudrait alors soit avoir au niveau du dispatcher un fichier stockant les numéros de série des paires produites et leur usine de production, ou bien que chaque usine connaisse les informations de l'ensemble des paires produites par les autres usines, ce qui serait dans ce cas beaucoup plus coûteux à mettre en place.

Conclusion

L'ensemble du groupe est très satisfait du résultat final de l'application obtenue et de l'architecture mise en place. Nous avons réussi à implémenter l'ensemble des fonctionnalités demandées, non sans peine et difficultés pour certaines. Ce projet nous aura permis de mettre en place un système distribué utilisant de nombreuses dépendances via Maven, de déployer nous même une dépendance grâce aux GitHub Actions en ligne ou bien encore de mettre en place des tests unitaires avec JUnit qui nous ont permis de tester la robustesse de notre application. Nous avons donc renforcé nos compétences et nos connaissances techniques dans un gros projet en équipe, où nous avons su communiquer et nous entraider quand il le fallait.